

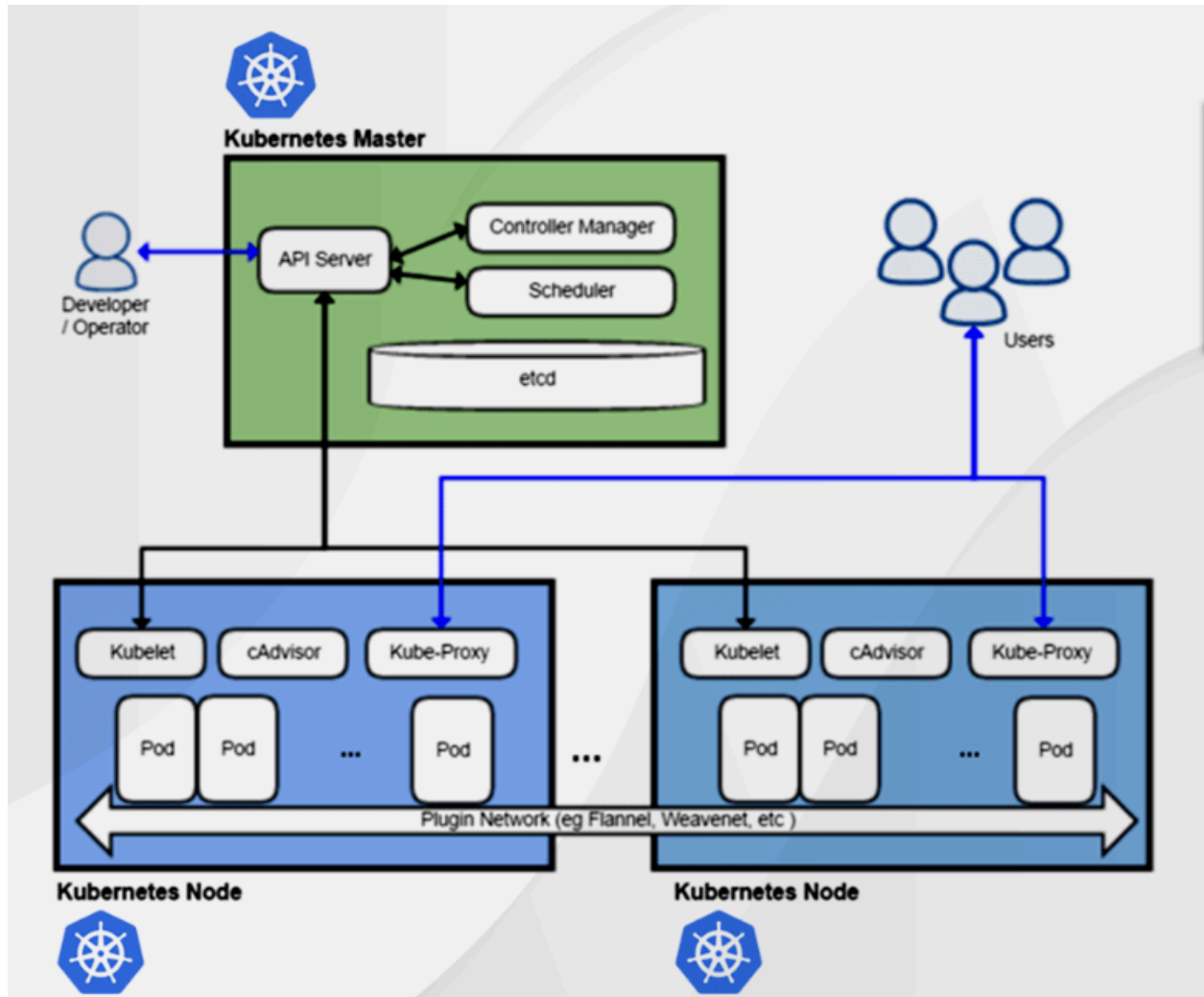
What is Kubernetes?

Kubernetes (often abbreviated as **K8s**) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. Initially developed by Google and now maintained by the **Cloud Native Computing Foundation (CNCF)**, Kubernetes helps developers and operators manage applications in a highly efficient and automated way across clusters of machines.

Kubernetes Architecture Components

Kubernetes has a **master-slave** architecture, where there is a **master** responsible for the management and orchestration of the cluster, and **nodes** (also called worker nodes) where the application containers run. The architecture is highly modular, scalable, and designed for high availability.

Key Components of Kubernetes Architecture:



1. Master Components (Control Plane)

The **Control Plane** is responsible for managing the Kubernetes cluster and making global decisions, such as scheduling and orchestration of workloads. It runs on one or more machines and controls the cluster's overall state.

Key Control Plane Components:

- **API Server (kube-apiserver):**
 - The central management point for communication between all parts of the cluster. It exposes the REST API, which is used by all the other components

(e.g., `kubectl`, nodes, etc.) to interact with the cluster.

- It handles requests for the cluster's state (e.g., deploying apps, scaling, etc.) and ensures consistency in the system by updating the state as needed.
 - **Scheduler (`kube-scheduler`):**
 - The scheduler is responsible for selecting which node will run a newly created pod. It takes into account factors like resource availability, resource requirements, node constraints, and other policies.
 - It runs on the master node and assigns pods to worker nodes.
 - **Controller Manager (`kube-controller-manager`):**
 - The controller manager is responsible for maintaining the desired state of the system, including monitoring the cluster for any changes, like node failures or resource shortages, and then adjusting accordingly.
 - It contains multiple controllers (e.g., Deployment Controller, ReplicaSet Controller, Node Controller) that handle various operational aspects like scaling or maintaining the health of pods.
 - **etcd:**
 - **etcd** is a distributed key-value store used to store all cluster data, such as configuration, state, and metadata. It is the source of truth for the current state of the Kubernetes cluster.
 - For high availability, **etcd** can be configured in a cluster with multiple replicas.
-

2. Node Components (Worker Nodes)

Each worker node in a Kubernetes cluster runs the applications and workloads. Worker nodes have a few critical components that help ensure the applications run smoothly.

Key Node Components:

- **Kubelet:**

- The **kubelet** is an agent that runs on each worker node. It ensures that the containers are running as expected by checking the state of the containers and reporting back to the master.
- It also manages the containers as per the desired state set by the master, making sure that the application containers (pods) are running as expected.

- **Kube Proxy:**

- The **kube-proxy** runs on every node and maintains network rules to allow communication between pods across nodes. It can perform load balancing and ensure that network traffic reaches the appropriate pod.
- It helps route requests to the right service endpoints based on IP and port.

- **Container Runtime:**

- The **container runtime** is responsible for running the actual containers. Kubernetes supports several container runtimes, like Docker, containerd, and others.
- It works closely with the kubelet to start and stop containers as needed.

- **Pods:**

- A **Pod** is the smallest deployable unit in Kubernetes and can contain one or more containers. Pods are often used to manage application containers that share the same network namespace and storage.
 - Every pod is scheduled and executed by the kubelet on a worker node.
-

3. Other Important Kubernetes Concepts

- **Namespaces:**
 - Namespaces are a way to divide cluster resources into logically named groups. This can be used to organize different environments (e.g., dev, staging, prod) or manage resource quotas across teams.
- **Services:**
 - A **Service** in Kubernetes is an abstraction layer that defines a set of pods and provides a stable endpoint for accessing those pods. This can be used for load balancing or for accessing pods in a consistent way, regardless of changes in pod IP addresses.
- **Deployments:**
 - A **Deployment** is a higher-level abstraction that manages stateless applications. It defines the desired state of the application, such as the number of replicas, and Kubernetes ensures that the actual state matches the desired state.
 - It can automatically roll out changes and roll them back if necessary.
- **ReplicaSets:**
 - **ReplicaSets** are used to maintain a stable set of replica pods running at any given time. They ensure that a specific number of replicas of a pod are running and available.
- **StatefulSets:**
 - **StatefulSets** are used for managing stateful applications that require persistent storage (e.g., databases). Unlike Deployments, StatefulSets preserve the identity and state of pods across restarts.
- **ConfigMaps and Secrets:**

- **ConfigMaps** are used to store configuration data that can be consumed by applications as environment variables or configuration files.
 - **Secrets** store sensitive information, such as passwords or API keys, in an encrypted format.
-

Kubernetes Cluster Architecture Diagram

In a typical Kubernetes cluster:

- The **Master node** (Control Plane) runs the API Server, Scheduler, Controller Manager, and etcd.
- The **Worker nodes** (Minions) run the kubelet, kube proxy, and containers in Pods.

The communication between the nodes and master happens through the API server, with the master making decisions about how the cluster should be run, while the nodes perform the work.

Summary

- **Control Plane (Master):** Manages the cluster and makes decisions about pod scheduling, scaling, and maintaining desired state.
- **Worker Nodes:** Run the application workloads, including containers inside Pods, and communicate with the master via the kubelet and kube proxy.
- **Pods:** The smallest unit of deployment in Kubernetes, typically containing one or more containers.
- **Services:** Abstraction that allows reliable communication between pods.

